Table of contents

Serie 2

Methods

Method 1: Syntax Analysis

- **Context-free grammar (CFG) verification**: identification of basic language structure errors (parentheses, punctuation, keywords, token order).
- **Contextual verification**: identification of errors that depend on context (undeclared variables, incompatible types, scope rules).

Method 2: Operational Semantics

- **Evaluation by transition rules**: use of valuation functions (`Val`) and memory (`Mem`) to describe step-by-step program execution.
- **Expression evaluation**: calculation of an expression's value in a given memory state.
- **Memory update**: modification of the memory state during assignment or other actions.

Method 3: Formal Specification with First-Order Logic (FOL)

- **Preconditions**: conditions on inputs that must be true before execution.
- **Postconditions**: conditions on outputs/memory that must be true after execution.
- **Formalization of properties**: use of quantifiers, logical and arithmetic operators to express invariants, goals.

Method 4: Data Structure Modeling (Arrays)

- **Array representation**: use of `a.length` and `a[i]` to manipulate arrays.
- **Array traversal**: iteration over indices with `while` or `for` loops.
- **Transformation specification**: informal description then formalization in FOL.

Reminder

- **Syntax/semantics distinction**: syntax concerns form, semantics concerns meaning.
- **Operational semantics**:
  - `Val(e, )` gives the value of expression `e` in memory state
  - `Mem(P, )` gives the memory state after executing program `P` from .
- **Logic laws**:
  - Implication elimination: `(P Q)` `(¬P Q)`
  - Distributivity: `P (Q R)` `(P Q)` `(P R)`, etc.
  - De Morgan: `¬(P Q)` `¬P` `¬Q`, `¬(P Q)` `¬P` `¬Q`
- **Preconditions/postconditions**: partial correctness (if the precondition is true and the program terminates, then the postcondition is true); termination must be treated separately.
- **Loop invariants**: property that remains true before and after each iteration, used to prove loop correctness.

Illustrative Examples

Example 1: Syntax and Contextual Analysis

Context: Exercise 1 – error detection in a `remainder` program.

Application:

- The given program is:

```
remainder(a,b) = aux := b; while (res >= aux) res := res - aux; done res;
```

- **Context-free grammar error**: the semicolon after `res := res - aux` is incorrect according to the described syntax (probably a misplaced instruction separator). Moreover, the `while` loop should have a delimited instruction block (e.g., with `do ... done`), but here a `do` is likely missing after the condition.
- **Contextual error**: the variable `res` is used but was never declared or initialized. Similarly, `aux` is declared but `res` is not. This violates a contextual rule (every variable must be declared before use).

Example 2: Operational Semantics with Valuation and Memory

Context: Exercise 2 – evaluation of `abs(a,b)` in state `= {a ← 3, b ← 9}`.

Application:

- We assume that `abs(a,b)` is defined as:

  ```
  abs(a,b) = res := 0; if (a < b) then res := b - a else res := a - b endif; res
  ```

- Step by step:

  1. `Val(abs(3,9), ) = Val(abs program; res, )`
  2. We evaluate the program: `Mem(abs program, )`
  3. `Mem(res := 0, ) = [res ← Val(0, )] = ' = {a ← 3, b ← 9, res ← 0}`
  4. We evaluate the `if`: `Val(a < b, ') = (Val(a, ') < Val(b, )) = (3 < 9) = T`
  5. So we execute `res := b - a`: `Mem(res := b - a, ') = '[res ← Val(b - a, ')] = '' = {a ← 3, b ← 9, res ← 6}`
  6. Then, we evaluate `res` in `''`: `Val(res, '') = 6`

- Thus, `Val(abs(3,9), ) = 6`.

Example 3: FOL Specification and Array Modeling

Context: Exercise 3 – function that fills `b` with the cumulative sums of `a`.

Application:

1. **Informal implementation**:

   ```
   sum_prefix(a, b) =
     i := 0;
     sum := 0;
     while (i < a.length) do
       sum := sum + a[i];
       b[i] := sum;
       i := i + 1;
     done
   ```

2. **Precondition**: `a.length = b.length  a.length > 0`

3. **Postcondition**: `j  [0, a.length-1], b[j] = Σ_{k=0}^{j} a[k]` In FOL, we could write:

   ```
   j (0  j  j < a.length  b[j] = sum_{k=0}^{j} a[k])
   ```

where `sum` is a metanotation for summation (in pure FOL, we use induction or a defined summation function).

4. **Semantic evaluation** for `= {a ← [0,3,5,6], b ← [0,0,0,0]}`:

   - Initially `i=0`, `sum=0`.
   - Iteration 1: `sum=0+0=0`, `b[0]=0`, `i=1`
   - Iteration 2: `sum=0+3=3`, `b[1]=3`, `i=2`
   - Iteration 3: `sum=3+5=8`, `b[2]=8`, `i=3`
   - Iteration 4: `sum=8+6=14`, `b[3]=14`, `i=4`
   - Final state: `' = {a ← [0,3,5,6], b ← [0,3,8,14], i ← 4, sum ← 14}`

Synthesis

The exercises implement several key methods in software verification: syntax analysis (grammar and context), operational semantics (step-by-step evaluation with memory states), and formal modeling with first-order logic and data structures (arrays). Mastering these methods enables detecting errors, precisely specifying expected behavior, and reasoning about program execution.